---

1. Giải pháp đầu tiên được biết đến trong việc giải quyết tranh chấp vùng critical-section cho 2 process bằng phần mềm là từ tác giả Dekker, với code như bên dưới (Trong slide học, một giải pháp phần mềm khác cũng giải quyết vấn đề critical-section cho 2 process là giải thuật của Peterson)
Hai process P_0 và P_1 dùng 2 biến chia sẻ:

boolean flag[2]; /* khởi tạo false */
int turn; /*chỉ nhận giá trị 0 hay 1; khởi tạo có thể là 0 hay 1*/

Code cho process *Pi* (i = 0 hoặc 1) như sau:

```
do {
   flag[i] = true;

   while (flag[j]) {
      if (turn == j) {
         flag[i] = false;
         while (turn == j)
            ; /* do nothing */
         flag[i] = true;
      }
   }

      /* critical section */

   turn = j;
   flag[i] = false;

      /* remainder section */
} while (true);
```

Giải thuật trên thỏa mãn 3 yêu cầu khi giải quyết bài toán critical-section không? Tại sao

Đáp án:

Thỏa mãn cả 3 yêu cầu

Với từng yêu cầu trong 3 yêu cầu khi giải quyết bài toán critical-section, sinh viên giải thích tại sao từng yêu cầu này thỏa với giải thuật trên.

2.
Xét giải pháp đồng bộ hóa sau:

```
while (TRUE) {
    int j = 1- i;                    /*i = 0 thì j bằng 1 và ngược lại*/
    flag[i]= TRUE;
    turn = i;
    while (turn == j && flag[j] == TRUE);
    Critical-section;
    flag[i] = FALSE;
    Remainder-section;
}
```

Với flag và turn là 2 biến chia sẻ

```
boolean flag[2];
int turn;
```

Và đoạn code trên chạy ở hai process Pi và Pj

Giải pháp này có đảm bảo cả ba yêu cầu khi giải quyết bài toán critical-section hay không?

Đáp án:

Không.

Xét tình huống khi flag[0] = true; turn = 0 ➔ P0 vào critical-section, nếu lúc đó flag[1] = 1, P1 có thể gán turn = 1 và vào luôn critical-section.

Các dạng bài tập bên dưới liên quan đến Semaphore.

Sinh viên chú ý: Thực tế, có nhiều thư viện hỗ trợ Semaphore, hàm wait và signal có thể khác đôi chỗ so với Semaphore học trong lý thuyết chương 5. Vì vậy, khi coi tài liệu thêm ở ngoài cần lưu ý điểm này.

Semaphore trong lý thuyết chương 5 theo quy tắc:

```
typedef  struct {
        int  value;
        struct  process *L; /* process queue */
} semaphore;
```

*void wait(semaphore *S) {* *S.value--;* *if (S.value < 0) {* *add this process to S.L;* *block();* *}* *}*	*void signal(semaphore *S) {* *S.value++;* *if (S.value <= 0) {* *remove a process P from S.L;* *wakeup(P);* *}* *}*

3.

Xét hai tiến trình sau (biết khởi tạo na = nb = 0)

Process A

```
while (TRUE)
{
    na = na + 1;
}
```

Process B

```
while (TRUE)
{
    nb = nb + 1;
}
```

a. Đồng bộ hóa xử lý của 2 tiến trình trên, sử dụng 2 semaphore tổng quát, sao cho tại bất kỳ thời điểm nào cũng có nb <= na <= nb +10.
b. Nếu giảm điều kiện chỉ có là na <= nb +10, giải pháp của bạn sẽ được sửa chữa như thế nào?
c. Giải pháp của bạn có còn đúng nếu có nhiều tiến trình loại A và B cùng thực hiện?

Đáp án:

Đáp án gợi ý (sinh viên có thể giải bằng các cách khác nhau):

Khởi tạo 2 Semaphore Sa và Sb sao cho ban đầu value của Sa bằng 0 và value của Sb bằng 10

Process A

```
while (TRUE)
{
    wait(Sb);
    na = na + 1;
    signal(Sa);
}
```

Process B

```
while (TRUE)
{
    wait(Sa);
    nb = nb + 1;
    signal(Sb);
}
```

5.

Một biến X được chia sẻ bởi 2 tiến trình cùng thực hiện đoạn code sau:

```
do
    X = X +1;
    if ( X == 20) X = 0;
while ( TRUE );
```

Bắt đầu với giá trị X = 0, chứng tỏ rằng giá trị X có thể vượt quá 20. Cần sửa chữa đoạn chương trình trên như thế nào để đảm bảo X không vượt quá 20 (dùng Semaphore)?

Đáp án:

Đáp án gợi ý:

Tạo một Semaphore Mutex, khởi tạo giá trị Value bằng 1

```
while ( TRUE )
{
    wait(Mutex);
    X = X + 1;
    if ( X == 20) X = 0;
    signal(Mutex);
}
```

6.

a. Xét 2 process P1 và P2 thực thi đồng thời, với P1 {A1, A2 }; P2 {B1, B2 } (tức P1 phải hoàn tất lần lượt 3 lệnh A1, A2; P2 phải hoàn tất lần lượt 2 lệnh B1, B2; và các lệnh này đều là đơn nguyên (atomic)) Dùng Semaphore để đồng bộ hóa hoạt động của 2 tiến trình này sao cho A2 và B2 được thực thi chỉ sau khi cả A1 và B1 hoàn tất.

b. Tổng quát hóa câu a cho các tiến trình có đoạn chương trình sau:

Process P1 { for (i = 1; i <= 100; i ++) A_i }

Process P2 { for (j = 1; j <= 100; j ++) B_j }

Đồng bộ hóa hoạt động của 2 tiến trình này sao cho với *k* bất kỳ (2 <= k <= 100), A_k chỉ có thể bắt đầu khi $B_{(k-1)}$ đã kết thúc và B_k chỉ có thể bắt đầu khi $A_{(k-1)}$ đã kết thúc.

Đáp án:

a. Đáp án gợi ý:

Dùng 2 Semaphore AB và BA để đồng bộ. Hai Semaphore này đều được khởi tạo với Value = 0;

Process P1

```
{
    A1;
    signal(BA);
    wait(AB);
    A2;
}
```

Process P2

```
{
    B1;
    signal(AB);
    wait(BA);
    B2;
}
```

b. Đáp án gợi ý:

- Nếu P1 và P2 được viết tách rời các Ai và Bj ra như bên dưới (**không dùng for**), dùng các Semaphore từ SA1 tới SA99 và từ SB1 tới SB99 (Value của tất cả các Semaphore này **khởi tạo bằng 0**) như sau:

Process P1

```
{
    A1;
    signal(SA1);
    wait(SB1);
    A2;
    signal(SA2);
    wait(SB2);
    A3;
    signal(SA3);
    …
    …
    wait(SB99);
    A99;
    signal(SA99);
    wait(SB99);
    A100;
}
```

Process P2

```
{
    B1;
    signal(SB1);
    wait(SA1);
    B2;
    signal(SB2);
    wait(SA2);
    B3;
    signal(SB3);
    …
    …
    wait(SA98);
    B99;
    signal(SB99);
    wait(SA99);
    B100;
}
```

- Nhưng nếu P1 và P2 được **viết dùng for** như câu hỏi, thì chỉ dùng 2 Semaphore AB và BA (lưu ý: Value của 2 Semaphore này phải được **khởi tạo bằng 1**)

Process P1

```
{
    for ( i = 1; i<=100; i++)
    {
        wait(AB);
        Ai;
        signal(BA);
    }
}
```

Process P2

```
{
    for ( i = 1; i<=100; i++)
    {
        wait(BA);
        Bi;
        signal(AB);
    }
}
```

7.

Biết các biểu thức từ (1) tới (7) được chạy lần lượt trong các process từ P1 tới P7 với các biến dùng chung. Sử dụng semaphore để đảm bảo các biểu thức tính toán đúng giá trị.

w = x1 * x2 (1)

v = x3 * x4 (2)

y = v * x5 (3)

z = v * x6 (4)

y = w * y (5)

z = w * z (6)

ans = y + z (7)

Đáp án:

Nhận xét thứ tự thực hiện:

(3) phải chờ (2) thực hiện xong thì mới được thực hiện (vì chờ nhận được giá trị v mới cập nhật)

➔ Dùng Semaphore S1 để đồng bộ

(4) phải chờ (2) thực hiện xong thì mới được thực hiện (vì chờ nhận được giá trị v mới cập nhật)

➔ Dùng Semaphore S2 để đồng bộ

(5) phải chờ cả (1) và (3) thực hiện xong thì mới được thực hiện

➔ Dùng 2 Semaphore S3 và S4 để đồng bộ

(6) phải chờ cả (1) và (4) thực hiện xong thì mới được thực hiện

➔ Dùng 2 Semaphore S5 và S6 để đồng bộ

(7) phải chờ (5) và (6) thực hiện xong thì mới được thực hiện

➔ Dùng 2 Semaphore S7 và S8 để đồng bộ

Các Semaphore từ S1 tới S8 đều được khởi tạo Value = 0

Đáp án:

Process P1
{
w = x1 * x2;
signal(S3);
signal(S5)
}
	Process P5
	{
	wait(S3);
	wait(S4);
	x = w * y;
	signal(S7);
	}

Process P2
{
v = x3 * x4;
signal(S1);
signal(S2);
}
	Process P6
	{
	wait(S5);
	wait(S6);
	z = w * z;
	signal(S8);
	}

Process P3
{
wait(S1);
y = v * x5;
signal(S4);
}
	Process P7
	{
	wait(S7);
	wait(S8);
	ans = y + z;
	}

Process P4
{
wait(S2);
z = v * x6;
signal(S6);
}	